Smart App PH: System Paper (MVP Edition)

An installable ordering app and owner dashboard for small Philippine businesses
Webserbisyo · Prepared for: Alex (founder and developer) · Version 1.1 · October 12, 2026
This paper is the foundation for building Smart App PH a second time. The first build tried to ship every feature at once and stopped. This edition defines a small launch scope that works end to end. The design stays ready for the full product, which is listed in section 11. It is written like a capstone system paper: the problem, the objectives, the method, the system design, the data design, the screens and the plan to test and launch.

Contents

1. Introduction
2. Review of existing solutions
3. Methodology
4. Scope of the launch version (MVP)
5. Users and roles
6. System flows
7. Data design
8. Architecture, stack and infrastructure
9. User interface and experience
10. Security, privacy and reliability
11. Roadmap after launch
12. Business model
13. Testing and acceptance
14. Risks and limitations
15. Key decisions
16. Appendix A: Founder operations by SQL
17. Appendix B: Demo data
18. Appendix C: Glossary

1. Introduction

1.1 Background

Small Philippine businesses such as cafés, milk tea shops, bakeries, home cooks, clothing sellers and online shops take most of their orders through Facebook Messenger, phone calls and walk-ins. Orders get lost in long chat threads, payments arrive as GCash screenshots that are hard to match, and the owner has no list of regular customers to invite back. Delivery platforms solve ordering but charge a commission per order and keep the customer relationship to themselves.
Smart App PH gives each shop its own installable ordering app (a progressive web app, or

PWA) and a simple dashboard. Customers order from the shop's own link or QR code, pay the shop directly, and join the shop's stamp card. The owner sees every order in one place and builds a member list they can reach for free. The pitch to owners: **"May app ka na, may website ka pa."**

1.2 Problem statement

1. Orders from Messenger, calls and walk-ins are not in one place, so they get missed or mixed up.
2. GCash and Maya payments are confirmed by screenshot, with no reference check, which invites fake or reused proofs.
3. Owners have no customer list and no loyalty program, so one-time buyers do not come back.
4. Commission-based platforms take a cut of every order, and the shop does not own the customer.
5. Building a custom app is too expensive for a small shop.

1.3 General objective

To build and launch a multi-shop ordering platform where each small business gets an installable storefront app and an owner dashboard. Orders, payments and members live in one place, at a flat monthly fee with no commission.

1.4 Specific objectives (launch version)

1. Let customers browse, order and pay from a fast, installable mobile app with a bottom navigation bar, without creating an account.
2. Support the three most common order types: walk-in (ordering inside the shop), pickup and delivery.
3. Record every order with a gap-free daily order number and a digital receipt with a QR code for tracking.
4. Let customers pay in cash, by cash on delivery, or by GCash/Maya/bank transfer with an uploaded proof and a reference number, which the owner verifies.
5. Continue the conversation on Messenger with the order details already filled in.
6. Let customers join as members with an email code, earn stamps and redeem a reward.
7. Give the owner one dashboard for orders, products and stock, members, basic sales numbers and shop settings (logo, name, hours, colours).
8. Keep every shop's data separate and secure, so that adding more shops and features later needs no rewrite.

1.5 Scope and limitations

In scope for launch: the storefront PWA, the owner dashboard (one owner per shop), the order types walk-in, pickup and delivery, manual payment verification, members and stamps, product and stock management, basic reports, shop look (logo, cover, 60/30/10 colours), business hours, and Messenger hand-off.

Out of scope for launch (planned, see section 11):

- a point-of-sale (POS) screen, kitchen display, staff accounts and cash shifts
- promotions and vouchers
- dine-in table QR and shipping
- advance-order calendar
- push notifications
- full analytics and Google Sheets
- a platform admin screen
- the theme editor beyond colours
- payment gateways
- the native iOS and Android dashboard app

Limitations:

- Payments are verified by the owner, not by a payment gateway.
- Order slips are not BIR official receipts.
- iPhone users must add the app to the home screen by hand.
- The founder approves shops and tracks subscriptions with SQL in the Supabase dashboard, not in a custom admin screen.

1.6 Significance

- **Shop owners:** fewer missed orders, safer payment checks, a member list they own, and a professional app at a monthly cost below one day of sales.
- **Customers:** a clear menu, an order number and live status, a receipt they can keep, and rewards for coming back.
- **Webserbisyo:** a product that one developer can run for many shops, on one codebase and one database.

2. Review of existing solutions

Solution	What it does well	Gap for our target shops
Food delivery platforms (GrabFood, foodpanda)	Huge reach, riders included	Commission on every order; the platform owns the customer; not for walk-in or pickup loyalty
Facebook page + Messenger	Free, every customer already uses it	No order list, no menu with prices, no payment check, orders lost in chat
POS apps (Loyverse and similar)	Strong counter selling and inventory	Built for the cashier, not for customers ordering from their own phone
Store builders (Shopify and similar)	Full online stores	Priced in dollars, built for shipping e-commerce, weak for walk-in, pickup and stamp

Solution	What it does well	Gap for our target shops
		cards
Smart App PH	The shop's own app, ordering + members + stamps, flat fee, PH payments, Messenger hand-off	No riders; manual payment verification at launch

3. Methodology

3.1 Development approach

The project uses an **iterative, MVP-first approach**: build the smallest version that works end to end, test it with real shops, then add one feature at a time. Lessons from the first build turned into rules:

What went wrong in the first build	Rule for this build
Every feature at once: POS, kitchen, staff, shifts, promos, reports	Launch scope only (section 4). New features wait in the roadmap.
The UI was redesigned three times	Decide the look once (section 9) and build every screen in it from the first day.
Six shared packages and unclear places for code made the repo hard to follow	Two apps (storefront and dashboard) and only three shared packages, with a fixed place for every kind of code (section 8.3).
Large prompts produced messy code	One small checkpoint per prompt. Test it, commit it, then move on.
Demo data cost many AI tokens to generate	Demo data lives in SQL inside the database and is reset with one command (Appendix B).
Docs drifted from the code	One source of truth per topic: this paper for the product, the code for types (Zod), migrations for the database.

3.2 Tools and roles in development

- **Claude (chat):** planning, research, prompts, review.
- **Claude Code:** repo setup, packages, database migrations, server logic, tests, commits.
- **Antigravity:** screens and checking them in a browser.
- **GitHub:** code. **Vercel:** hosting and preview deploys. **Supabase:** database, auth, storage, realtime. **Resend:** email.

3.3 Development phases

The work runs in six phases, detailed in the companion document **Smart App PH Build Plan**:

1. repo and context
2. foundations
3. authentication
4. front end with mock data
5. back end and connections
6. performance and deployment

Each phase ends with a test by the founder using one food shop and one retail shop.

4. Scope of the launch version (MVP)

4.1 Feature list

Area	Launch features
Storefront app (customers)	● Shop home: logo, cover, open or closed status, categories, best sellers, members count, Messenger button. ● Menu or products with search. ● Item sheet: sizes or variants, add-ons, quantity, note. ● Cart, checkout, receipt with QR, live order status, my orders. ● Install prompt. Bottom navigation: Home, Menu, Orders, Account.
Order types	Walk-in (ordered inside the shop, paid at the counter), pickup (as soon as possible or at a chosen time today or tomorrow), delivery (address, landmark, zone fee or fee confirmed on Messenger). Each one is an owner switch.
Payments	Cash at the counter or on pickup, cash on delivery (owner switch), and GCash/Maya/bank transfer. For transfers the customer sees the shop QR and account number with a Copy button, uploads the proof (compressed automatically) and enters the reference number. The owner verifies and approves.
Messenger	After an order, "Send order to our Messenger" opens the shop's Messenger with the order details filled in, with "Copy order details" as a fallback.
Members	Guest checkout by default. Join with an email

Area	Launch features
	code after an order. A stamp card with a rule set by the owner, stamps added on completed orders, the reward redeemed at checkout. Member page with stamps, order history and a saved address.
Owner dashboard	• **Home:** today's sales and orders, to-verify and waiting counts, members card. • **Orders:** a board with one next-step button per order, a new-order chime, an order detail with the proof viewer. • **Products:** categories, items, variants, add-ons, photos, stock, sold out. • **Customers:** members marked. • **Reports:** today, 7 and 30 days; best sellers; CSV export. • **Shop settings.**
Shop settings	Name, logo, cover, short description, contact, address, Facebook page and Messenger, business hours and closed days, pause orders, order types, delivery zones, payment accounts with QR images, stamp card rule, colours (60/30/10).
Platform (founder)	Owner sign-up creates a pending shop. The founder approves, suspends and records subscription payments with SQL (Appendix A).

4.2 Not in the launch version

These features keep their place in the design and come back after launch (section 11):

- POS and cash shifts, kitchen display, staff accounts
- promotions
- dine-in table QR, shipping, advance-order calendar
- push notifications, announcements
- full reports, Google Sheets
- admin screen
- theme and layout editor
- PayMongo, Meta Pixel
- the native iOS and Android dashboard app, services and bookings

5. Users and roles

User	Uses	Signs in with	Can do
Guest customer	Storefront	Nothing (name and mobile number, remembered on the device)	Browse, order, pay, upload proof, track the order, keep the receipt
Member	Storefront	Email and a 6-digit code	Everything a guest can, plus stamps, rewards, order history on any device, saved address
Owner	Dashboard	Email and password (email confirmed by code)	Everything in their own shop; nothing in other shops
Founder	Supabase dashboard (SQL editor)	Supabase account	Approve or suspend shops, record subscription payments, monitor usage
Staff (later)	Dashboard	Username and shop code	Cashier or kitchen/packer permissions; the roles column and permission checks exist from day one

Words on screen follow the shop's type: a food shop sees Menu, Menu item and Kitchen; a retail shop sees Products, Product and Packing. The code uses neutral words (item, catalog, prep) and never checks the shop type to decide behaviour. Features are switches.

6. System flows

6.1 Shop onboarding

```
Owner                          System                              Founder
-----                          ------                              -------
Sign up (email, password) --> sends 6-digit code (Resend)
Enter code                --> account confirmed
Shop setup: type, name,   --> creates shop (status = pending)
  link name (slug)            "Waiting for approval" screen
                                                                   Runs approve SQL (Appendix
A)
                          <-- status = active, paid_until set
Opens dashboard: setup checklist (logo, hours, payment account,
  first product, order types) --> shares shop link and QR
```

6.2 Order types in real-life terms

Type	Real-life situation	Payment options	Status path
Walk-in	The customer is inside the shop, scans the QR on the counter, orders from their phone and waits for their number to be called	Cash at the counter, or GCash/Maya with proof	Placed → Preparing → Ready → Completed
Pickup	The customer orders ahead and picks up at a chosen time (as soon as possible, or a slot within business hours today or tomorrow)	GCash/Maya/bank with proof, or pay on pickup if the owner allows	Placed → Preparing → Ready → Completed
Delivery	The customer orders to an address. The owner books a rider (Lalamove, Grab or own rider) and can paste a tracking link	GCash/Maya/bank with proof, or cash on delivery if the owner allows	Placed → Preparing → Ready → Out for delivery → Completed

End states for every type: **Cancelled** (with a reason) and **Expired** (unpaid after the time limit). Nothing is deleted.

6.3 Customer ordering flow

```
Shop link / QR
   |
Shop home --> Menu --> Item sheet (variant, add-ons, qty, note) --> Add to cart
   |
Cart (edit qty, remove, use reward if member)
   |
Checkout
   1. How: Walk-in | Pickup (ASAP / time) | Delivery (address, landmark, zone)
   2. Who: name, mobile (+ email optional); remembered on this device
   3. Pay: Cash/COD (if allowed) | GCash/Maya/Bank (QR, Copy number, exact amount)
   4. Consent line (privacy) --> Place order
   |
Receipt page /shop/o/{token}
   - order number #042 + QR code of this page
   - items, total, payment status, live status
   - [Upload proof] (if GCash/Maya/Bank and not yet uploaded)
   - [Send order to our Messenger]  [Save receipt]  [Join to earn stamps]
```

6.4 Payment verification flow

```
Customer uploads proof + reference no.
```

```
   --> image compressed in the browser (WebP, about 200 KB)
   --> stored privately, order payment_status = pending ("To verify")
Owner dashboard shows "1 to verify" (chime)
   --> owner opens order, sees proof + reference, checks own GCash/Maya app
   --> Approve: payment confirmed, order paid
       Reject (reason): customer sees "Payment not accepted" and can upload again
Same reference number already used in this shop --> blocked before saving
```

Rule shown to the owner: **a screenshot never marks an order paid; check the reference number in your own wallet app.**

6.5 Owner order handling

```
New order arrives (realtime + chime)
  Orders board columns:  New | Preparing | Ready | Out | Done today
  Each card: #042 · type icon · name · items · total · payment chip · age
  One next-step button:
     To verify   -> [Verify payment]
     New (paid or pay-later allowed) -> [Start]
     Preparing   -> [Mark ready]
     Ready       -> pickup/walk-in: [Complete] (or [Mark paid] first if unpaid)
                    delivery: [Out for delivery] (+ tracking link) -> [Complete]
  More: Cancel (reason), Print slip, Call / Message customer
```

The receipt QR code links to the order page. At the counter the owner can type the order number or scan the customer's QR with the phone camera to open the order and complete it.

6.6 Messenger hand-off

The owner saves their Facebook page username. After placing an order, the customer taps "Send order to our Messenger", which opens *m.me/username* with a prefilled message:

```
Hi! Order #042 (Pickup 3:30 PM)
2x Iced Latte (Large) - P300
1x Ensaymada - P45
Total: P345 · GCash (proof uploaded)
Track: shop-link/o/abc123
```

The order is already saved, so Messenger is only for chatting. If the message text does not carry over, the customer uses "Copy order details".

6.7 Guest and member flow

```
Guest order (name + mobile) --> receipt page: "Join to earn stamps"
   --> enter email --> 6-digit code --> member
   --> past orders with the same mobile number are linked; stamps for completed orders added
Member next visit --> signed in --> checkout prefilled --> reward ready? apply at cart
```

- **Stamp rule** (owner sets): 1 stamp per completed order of at least the minimum amount; N stamps earn the reward (a peso discount or a free item shown as a discount).
- A stamp is added only when the order is completed and paid, and removed again if the order is later cancelled.
- **Members count** is shown on the owner's Home and, optionally, on the storefront as social proof ("120 members").

6.8 Stock flow

- Stock tracking is a switch per item. When it is on, each variant has a stock count.
- Placing an order reserves stock, and cancelling or expiring an order returns it.

- At 0, the variant shows as sold out automatically.
- The owner can also set sold out by hand, for example when an item ran out today.
- A low-stock number per variant shows a warning on Home.

6.9 Order lifecycle (state diagram)

```
                +-----------+  expire (unpaid, time limit)   +---------+
   place  ----> |  PLACED   | ----------------------------> | EXPIRED |
                +-----------+                               +---------+
                   |  start (paid, or pay-later type)
                   v
                +-----------+
                | PREPARING |
                +-----------+
                   |  mark ready
                   v
                +-----------+  out for delivery   +-----+
                |   READY   | ------------------> | OUT |
                +-----------+                     +-----+
                   | complete                        | complete
                   v                                 v
                +-----------+ <-----------------------+
                | COMPLETED |
                +-----------+
   cancel (with reason) is allowed from PLACED, PREPARING, READY, OUT --> CANCELLED

Payment status (separate): UNPAID -> PENDING (proof uploaded) -> PAID ; PENDING -> UNPAID
(rejected)
```

7. Data design

7.1 Rules

- Every shop-owned table has **tenant_id** (the shop) and row-level security, so a shop can only ever read its own rows.
- Money is stored as **whole centavos** (integer), never decimals. ₱120.00 is stored as 12000.
- Times are stored in UTC and shown in Manila time. The order's business date is the Manila date.
- Closed lists (order status, order type, payment method) are text with a check constraint, mirrored by Zod lists in the code.
- Order lines copy the item name and price at the time of the order, so editing or deleting a product never changes old receipts.
- Money, stock and stamps change only through database functions that check permissions. Simple owner edits (products, settings) use row-level security write rules.

7.2 Entity overview

```
tenants 1--* tenant_members *--1 profiles (auth users)
tenants 1--1 shop_settings ; 1--* business_hours ; 1--* closures
tenants 1--* payment_accounts ; 1--* delivery_zones
tenants 1--* categories 1--* items 1--* item_variants
                        items 1--* item_photos
```

```
                        items *--* option_groups (via item_option_groups) 1--* options
tenants 1--* customers 1--* consents ; customers 1--* customer_addresses
tenants 1--* orders 1--* order_items ; orders 1--* order_events ; orders 1--* payments
customers 1--* orders ; customers 1--* loyalty_ledger ; tenants 1--1 loyalty_rules
tenants 1--* order_counters (per day) ; tenants 1--* audit_log
```

7.3 Tables (launch version)

Table	Main columns	Notes
tenants	id, slug (unique), name, business_type (food, retail), status (pending, active, suspended), plan, paid_until, is_demo, created_at	Status and paid_until are changed by the founder with SQL
profiles	id (= auth user), display_name, phone	Filled by a trigger on sign-up
tenant_members	tenant_id, user_id, role (owner; later cashier, prep), status	One owner per shop at launch
shop_settings	tenant_id, logo_path, cover_path, tagline, phone, email, address, city, maps_url, facebook_url, messenger_username, color_primary (60), color_secondary (30), color_accent (10), accepting_orders, busy_until, walk_in_enabled, pickup_enabled, delivery_enabled, cod_enabled, pay_later_pickup, delivery_fee_via_messenger, unpaid_expiry_minutes, show_member_count	One row per shop, created with the shop
business_hours	tenant_id, weekday (0-6), opens_at, closes_at	Overnight hours allowed
closures	tenant_id, closed_on, note	Holidays and days off
payment_accounts	tenant_id, method (gcash, maya, bank), account_name, account_number, bank_name, qr_path,	Shown at checkout

Table	Main columns	Notes
	is_active, sort	
delivery_zones	tenant_id, name, fee, eta_text, is_active, sort	Flat fee per area
categories	tenant_id, name, sort, is_hidden	
items	tenant_id, category_id, name, description, status (active, hidden, archived), is_featured, track_stock, sort	Archived instead of deleted when it has orders
item_photos	tenant_id, item_id, path, sort	First photo is the cover
item_variants	tenant_id, item_id, name, price, stock_on_hand, low_stock_at, is_sold_out, sort, archived_at	Every item has at least one variant; price and stock live here
option_groups, options, item_option_groups	group: name, min_select, max_select; option: name, price_delta, is_sold_out	Add-ons such as sugar level or toppings
customers	tenant_id, user_id (null for guests), name, phone (+639…), email, member_since, order_count, total_spent, last_order_at, notes, tags	One row per person per shop; a member has member_since set
consents	tenant_id, customer_id, channel (privacy, sms, email, push), granted, source, created_at	Append-only; the newest row per channel wins
customer_addresses	tenant_id, customer_id, label, address_line, landmark, is_default	For members
orders	tenant_id, order_no, business_date, token (secret link), fulfillment (walk_in, pickup, delivery; later dine_in, shipping), status, payment_status, payment_method, customer_id,	total = subtotal - discount_total + delivery_fee

Table	Main columns	Notes
	customer_name, customer_phone, address_line, landmark, delivery_zone_id, delivery_fee, pickup_at, notes, subtotal, discount_total, total, tracking_url, cancel_reason, client_key, placed_at, paid_at, ready_at, completed_at, cancelled_at	
order_items	tenant_id, order_id, item_id, variant_id, item_name, variant_name, unit_price, options (list of chosen add-ons with prices), options_total, qty, line_total, note	Snapshots at order time
order_events	tenant_id, order_id, from_status, to_status, actor (customer, owner, system), note, created_at	Timeline shown on the receipt and in the dashboard
order_counters	tenant_id, business_date, last_no	Gap-free daily numbers; no direct access
payments	tenant_id, order_id, method, amount, status (pending, confirmed, rejected), reference_no, reference_norm, proof_path, reviewed_at, reject_reason	Reference unique per shop; several rows per order allowed (deposits later)
loyalty_rules	tenant_id, is_active, min_order_total, stamps_needed, reward_label, reward_amount	The stamp card
loyalty_ledger	tenant_id, customer_id, order_id, delta, reason (earned, redeemed, reversed, manual), created_at	Append-only; balance = sum of delta
audit_log	tenant_id, actor_id, action, target_type, target_id, meta,	Append-only record of owner actions

Table	Main columns	Notes
	created_at	

File storage:

- bucket *shop-media* (public): logos, covers, product photos, payment QR images
- bucket *payment-proofs* (private): customers' proofs, viewable by the shop owner only, deletable after 30 days

Files are stored under the shop's id, for example *tenant-id/items/photo.webp*.

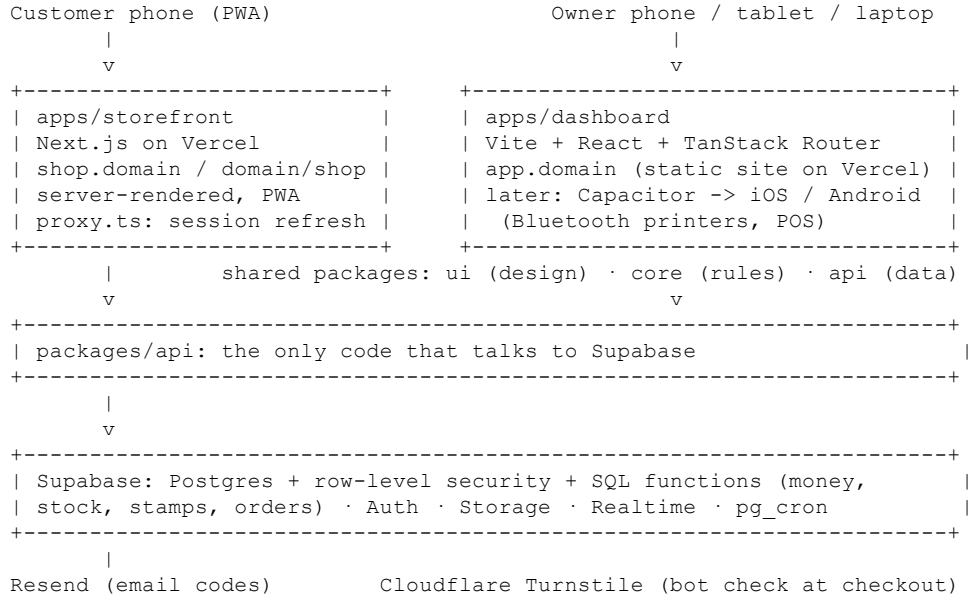
7.4 Database functions (server logic)

Function	Called by	What it does
create_my_shop	Owner (setup)	Creates tenant (pending), membership, settings, default hours
get_storefront(slug)	Anyone	Shop info, open status, categories, items, variants, add-ons, payment accounts, zones, members count
quote_order(payload)	Anyone	Recomputes prices, fees, reward and stock from the database before placing
place_order(payload)	Anyone	Validates, reserves stock, assigns the order number and token, creates or updates the customer, records the event; a retry with the same client_key returns the same order
submit_payment_proof(token, ...)	Anyone with the order link	Saves the proof and reference, blocks duplicates, sets payment to pending
get_order(token)	Anyone with the order link	Receipt and live status
review_payment, record_payment	Owner	Approve or reject a proof; mark a cash payment
advance_order, cancel_order	Owner	Moves the status by the allowed transitions; cancel returns stock and stamps
join_as_member,	Signed-in customer	Links the customer row (by

Function	Called by	What it does
get_my_account		phone or email) to the login, sets member_since, returns stamps and orders
get_dashboard_today, get_member_insights, get_sales_report	Owner	Home numbers, members card, basic reports
expire_unpaid_orders	Scheduled job (pg_cron)	Expires unpaid orders past the time limit and returns stock

8. Architecture, stack and infrastructure

8.1 Architecture



Why two apps:

- The storefront needs fast server-rendered pages that Google can find, which is what Next.js is for.
- The dashboard is an app that owners keep open all day. It will later ship to the iOS App Store and Google Play with Capacitor, for Bluetooth receipt printers and the POS. Capacitor needs a plain static app, which is what Vite builds.
- Separating them now avoids moving the dashboard out of Next.js later, right when printers and the POS arrive.
- Both apps share the same three packages, so the design, rules and data code are written once.

8.2 Stack

Layer	Choice	Notes
Storefront app	Next.js 16.3.8 or newer patch, App Router, React 19.3	Never below 16.3.8 (security fixes); *proxy.ts* replaces middleware; params are async
Dashboard app	Vite 8 + React 19.3 + TanStack Router (code-based routes) + TanStack Query	Builds to a static app; Capacitor wraps it later for iOS and Android
Native app (later)	Capacitor 8	Bluetooth receipt printers, POS, camera QR scan, push
Monorepo	pnpm workspaces + Turborepo	Runs both apps and the checks together
Language	TypeScript 6.0.3, strict	No *any*, no casts to silence errors
Validation and types	Zod 4	One schema per shape; types come from the schemas; database types generated by Supabase
Data fetching	TanStack Query 5 (hooks in packages/api)	Dashboard lists, live updates, optimistic buttons
Styling	Tailwind CSS 4 (CSS-first), shadcn/ui components, Tabler icons	One tokens file for colours, radius, fonts, shared by both apps
Forms	React Hook Form + Zod resolver (dashboard); Zod-checked forms in the storefront checkout	The same Zod schema checks the form and the API call
Images	Compression in the browser to WebP before upload (about 1600 px and 400 px sizes)	Used for product photos, logo, cover and payment proofs
QR codes	qrcode.react (show), camera scan later	Receipt QR, shop QR
Backend	Supabase (Postgres, Auth, Storage, Realtime, pg_cron)	Publishable key in the apps; secret key only in scripts and Edge Functions. Storefront

Layer	Choice	Notes
		sessions use cookies (@supabase/ssr); the dashboard uses the browser client
Email	Resend through Supabase Auth SMTP	Required before launch; built-in Supabase email is rate-limited
Bot protection	Cloudflare Turnstile + rate limit on place_order	Free
Hosting	Vercel, two projects (storefront, dashboard)	Hobby while building; Pro before the first paying shop
Tooling	pnpm, ESLint 10, Vitest, Supabase CLI	*pnpm check* = typecheck + lint + test

8.3 Folder structure and where code goes

The repo is split into a **front end** (the two apps and the shared design), **business logic** (rules and types), **back-end access** (the data layer) and the **back end** itself (the database).

```
smart-app-ph/
  apps/                          FRONT END: screens only
    storefront/                  Next.js customer app (PWA)
      src/app/[shop]/            routes: home, menu, i/[item], cart, checkout,
                                 o/[token], orders, account
      src/features/area/         components/, hooks/ for menu, cart, checkout, member
      proxy.ts                   session refresh
    dashboard/                   Vite owner app (Capacitor later)
      src/app/                   router, guards, layout shell, registry
      src/features/area/         screens/, components/, hooks/, module.ts
                                 areas: home, orders, products, customers,
                                 reports, settings, auth
  packages/
    ui/                          SHARED DESIGN: tokens.css, fonts, icons,
                                 shadcn components, shared patterns
                                 (cards, badges, empty states, uploaders)
    core/                        BUSINESS LOGIC: Zod schemas and types, money,
                                 order statuses and next actions, open/closed
                                 hours, vocab (Menu or Products), feature
                                 switches. No React, no Supabase.
    api/                         BACK-END ACCESS: Supabase clients, queries and
                                 mutations per area, TanStack Query hooks,
                                 image upload + compression, generated DB types,
                                 mock data for the front-end phase
  supabase/                      BACK END
    migrations/                  tables, RLS, SQL functions (orders, payments,
                                 stock, stamps); never edited after applied
    seed.sql                     demo shops and products
    demo.sql                     demo_reset() function
    functions/                   Edge Functions (later: emails, printer relay)
  docs/                          system paper, decisions, prompts
  CLAUDE.md                      rules for AI agents
```

You are writing	It goes in
A screen or page	*apps/dashboard/src/features/area/screens*, or the *src/app* routes of the storefront
A component used by one area	*apps/app-name/src/features/area/components*
A component used by both apps (button, card, status badge, uploader)	*packages/ui*
A rule (totals, next order step, open or closed, stamp progress)	*packages/core*
A shape of data (order, item, customer) and its type	*packages/core* as a Zod schema
A database read or write, or a data hook	*packages/api*
Anything about money, stock or stamps that must be trusted	A SQL function in *supabase/migrations*
A new table or column	A new migration file, with its RLS and grants

Rules:

- Apps never import each other.
- Only *packages/api* imports Supabase.
- *packages/core* has no React and no Supabase, so the same rules run in both apps, in tests and later in the native app.
- A dashboard feature is a folder with a *module.ts* (its routes and menu entry). A new feature is a new folder plus one line in the registry, never edits across the app.
- The lint config enforces these imports, so an AI agent can't break them by accident.

8.4 Infrastructure and growth path

Stage	Shops	Setup	Monthly cost (approx.)
Build and demo	0 paying	Vercel Hobby, Supabase Free	US$0
Launch	1 to about 100	Vercel Pro, Supabase Pro (daily backups), Resend free tier, one domain	About US$45 + domain
Growth	About 100 to 300	Bigger Supabase compute; images on Cloudflare R2 if storage grows	Grows with use

- **Storage:** each shop gets 500 MB. A product photo is about 200 KB after compression, so

even a 300-item clothing shop fits.
- **Domains:**
  - the dashboard at *app.platform-domain*
  - the storefront at *platform-domain/shop* at launch
  - a subdomain per shop (*shop.platform-domain*) as the next step
  - a custom domain as a paid add-on

9. User interface and experience

9.1 Principles

- Phone first. Touch targets of at least 44 px. No page scrolls sideways at 390 px width.
- One main action per screen; the next step is always obvious.
- Every status has an icon, a word and a colour, and each colour keeps one meaning (green paid or done, amber waiting or to verify, red late or problem).
- Every list has loading, empty and error states. Loading skeletons have the same shape as the content.
- Prices in pesos without .00 for whole amounts (₱120). Dates and times in Manila time.
- Build every screen once, in the final look below.

9.2 Look

- **Dashboard:**
  - navy sidebar on desktop (collapsible to icons)
  - bottom bar on phone
  - Plus Jakarta Sans font, Tabler icons
  - neutral grey hovers
  - marigold only for celebrations
  - a calm green for members
- **Storefront:** uses the shop's own three colours by the 60/30/10 rule.
  - 60% background and surfaces
  - 30% cards and secondary areas
  - 10% buttons and highlights

Text colours are picked automatically for contrast. The owner chooses from presets or picks colours, with a live phone preview.

9.3 Navigation

Surface	Phone	Desktop
Storefront	Bottom bar: Home · Menu · Orders · Account, with a floating cart bar above it	Top navigation, cart drawer on the right
Dashboard	Bottom bar: Home · Orders ·	Sidebar: Home, Orders,

Surface	Phone	Desktop
	Products · Customers · More (Reports, Settings)	Products, Customers, Reports, Settings; shop name and logo on top, account at the bottom

9.4 Wireframes (phone)

Storefront home

```
+---------------------------------+
| [logo] Juan's Cafe     (i) [=] |
| Open · until 9:00 PM           |
| +-----------------------------+ |
| |        cover photo          | |
| +-----------------------------+ |
| 120 members · Join for stamps  |
| [Coffee] [Milk tea] [Bread] >  |
| Best sellers                   |
| +-------+ +-------+ +-------+  |
| | photo | | photo | | photo |  |
| | Latte | | Wint. | | Ensay.|  |
| | P120 +| | P99  +| | P45  +|  |
| +-------+ +-------+ +-------+  |
| [ Message us on Messenger ]    |
| +-----------------------------+ |
| | Cart · 3 items      P345 > | |
| +-----------------------------+ |
| Home   Menu   Orders   Account |
+---------------------------------+
```

Item sheet

```
+---------------------------------+
| photo                      [x] |
| Iced Latte                     |
| Espresso, milk, ice            |
| Size (required, choose 1)      |
|  ( ) Regular P120              |
|  (o) Large   P150              |
| Add-ons (optional, up to 2)    |
|  [ ] Extra shot +P30           |
|  [ ] Oat milk   +P25           |
| Note: [less ice            ]   |
| [-] 1 [+]   [ Add · P150 ]     |
+---------------------------------+
```

Checkout

```
+---------------------------------+
| < Checkout                     |
| How do you want it?            |
| [Walk-in] [Pickup] [Delivery]  |
| Pickup time:                   |
|  (o) ASAP  ( ) 3:30 PM         |
| Your details                   |
| Name   [Maria Santos       ]   |
| Mobile [0917 123 4567      ]   |
| Payment                        |
|  (o) GCash ( ) Maya            |
|  ( ) Pay on pickup             |
| Subtotal P345 · Total P345     |
| [x] I agree to the privacy     |
|     notice                     |
| [        Place order        ]  |
```

```
+--------------------------------+
```

Receipt and status

```
+--------------------------------+
| Order #042         Pickup      |
| [QR CODE]  Show this at pickup |
| Status: Preparing  (live)      |
|  o Placed 3:05                 |
|  o Paid 3:07                   |
|  * Preparing                   |
| 2x Iced Latte (L)        P300  |
| 1x Ensaymada              P45  |
| Total                    P345  |
| GCash · Verified               |
| [Send order to our Messenger]  |
| [Save receipt] [Join stamps]   |
+--------------------------------+
```

Dashboard: orders (phone)

```
+--------------------------------+
| Orders          Open v  (bell) |
| [New 3][Prep 2][Ready 1][Out]  |
| +----------------------------+ |
| | #042  Pickup 3:30 · 4 min  | |
| | Maria S. (Member)          | |
| | 2x Iced Latte, Ensaymada   | |
| | P345   [To verify]         | |
| |          [Verify payment]  | |
| +----------------------------+ |
| | #043  Walk-in · 1 min      | |
| | Guest · 1x Wintermelon M   | |
| | P99    [Not paid]          | |
| |          [ Mark paid ]     | |
| +----------------------------+ |
| Home Orders Products Cust. More|
+--------------------------------+
```

Dashboard: home (desktop)

```
+------------+-------------------------------------------------------------+
| [logo] Shop| Good afternoon, Juan            [Share shop] [Shop QR]  |
| Home       | [2 to verify] [3 waiting] [1 low stock]                 |
| Orders  3  | +-----------+ +-----------+ +-----------+ +-----------+ |
| Products   | |Sales today| |Orders     | |Avg order  | |New members| |
| Customers  | |P3,322 +12%| |18 · 3 open| |P185       | |9 this week| |
| Reports    | +-----------+ +-----------+ +-----------+ +-----------+ |
| Settings   | Your members: 124 · Top suki: Maria P2,340 ...          |
|            | In progress (list)              | Best sellers today    |
| [avatar]   |                                 |                       |
+------------+-------------------------------------------------------------+
```

Dashboard: product editor

```
+-------------------------------------------+
| < New product                    [Save]   |
| Photos [+][img][img]                      |
|   (drag to sort; compressed on upload)    |
| Name     [Iced Latte        ]             |
| Category [Coffee v]                       |
| Variants                                  |
|  Regular  P[120]  Stock [  ]  Sold out [ ] |
|  Large    P[150]  Stock [  ]  Sold out [ ] |
|  [+ Add variant]                          |
| Track stock [on/off]   Low at [5]         |
| Add-on groups [Sugar level] [+]           |
| Visible in shop [on]                      |
+-------------------------------------------+
```

Settings: shop look

```
+-------------------------------------------+
| < Shop look                               |
```

```
| Logo [img] [Change]   Cover [img] [Change] |
| Name [Juan's Cafe   ]                      |
| Colours (60/30/10)                         |
|  Main 60%   [#FFFFFF]                      |
|  Cards 30%  [#F3EFE7]                      |
|  Accent 10% [#1F4FD8]                      |
|  Presets: (Uling) (Kape) (Ube) (Dagat)     |
| Preview: [phone mockup]                    |
+--------------------------------------------+
```

10. Security, privacy and reliability

- **Row-level security on every table**, written in the same migration that creates the table. Tests sign in as each test shop and prove one shop cannot read another shop's rows.
- The apps hold only the Supabase URL and the publishable key. The secret key lives only in scripts and Edge Functions.
- Server code checks the user with *getClaims()*; a session read from cookies is never trusted on its own.
- Prices, totals, rewards and stock are always recalculated in the database; the app's numbers are never trusted.
- **Order numbers have no gaps**; cancelled and expired orders stay visible with a reason.
- Payment proofs are private. A reference number can be used only once per shop.
- Checkout has Cloudflare Turnstile and a rate limit, against fake orders.
- Data Privacy Act:
    - a privacy notice per shop
    - a consent line at checkout
    - separate opt-ins for SMS, email and app alerts
    - customer data used only by that shop
- Order slips say "ORDER SLIP · NOT AN OFFICIAL RECEIPT"; shops keep issuing BIR receipts their usual way.
- Reliability:
    - a retry of the same checkout returns the same order (client key)
    - the dashboard falls back to checking every 15 seconds when realtime drops
    - Supabase Pro daily backups before the first paying shop

11. Roadmap after launch

Each item is added as new files and new migrations; nothing in the launch tables is renamed.

1. New-order email to the owner (Resend) and a QR scanner in the dashboard
2. Dine-in with a table QR (tables table, table on the order)
3. Shipping by region for online sellers
4. Promotions and vouchers (percent, peso, free item, free delivery, members-only)
5. Staff accounts: cashier and kitchen/packer, with permission checks already designed
6. Advance orders with a calendar and deposits
7. Push notifications and announcements to members
8. Native dashboard app for iOS and Android (Capacitor) with Bluetooth receipt printers
9. Counter POS, cash shifts, owner PIN for voids and discounts, kitchen display
10. Full reports, Google Sheets export, a platform admin screen, billing automation

11. Theme and layout editor, custom domains, PayMongo, Meta Pixel and Conversions API, a services and bookings type

12. Business model

Item	Value
Early "Pro" plan	**₱599 per month per shop**: storefront app, dashboard, members and stamps, 500 MB storage, no commission
Fixed tool cost at launch	About US$45 per month (Vercel Pro + Supabase Pro) plus the domain
Break-even on tools	About 5 paying shops
Later add-ons	Setup service, custom domain, extra storage, staff seats, custom website
Billing at launch	The shop pays by GCash, Maya or bank. The founder records the payment with SQL, which extends paid_until (Appendix A).

Subscription states, enforced later by a scheduled job:

- **Active**
- **Overdue**, days 1 to 7: everything works, with a banner
- **Restricted**, days 8 to 30: orders work, edits are locked
- **Paused**, after day 30: the storefront shows the menu and a Messenger button only

13. Testing and acceptance

Every phase is tested with two seeded shops: **Juan's Cafe** (food) and **Tala Apparel** (retail), each with demo products, members and orders. The launch version is accepted when all of these pass on a real phone:

1. A new owner signs up, confirms the email code, sets up a shop and sees "Waiting for approval". After the approve SQL, they reach the dashboard.
2. The owner adds a product with two variants, a photo (compressed) and stock of 3; it appears in the storefront within seconds.
3. A guest places a walk-in order paid in cash. The owner marks it paid, starts it, marks it ready and completes it. The receipt page updates live at each step.
4. A guest places a pickup order with GCash, uploads a proof and enters a reference number. The owner verifies it. A second order with the same reference is blocked.
5. A delivery order with cash on delivery goes Out for delivery with a tracking link, then Completed.
6. The guest joins with an email code. Their completed orders add stamps, and at the target the reward applies at checkout.
7. Stock reaches 0 and the variant shows sold out. A cancelled order returns the stock.

8. An unpaid GCash order expires after the time limit.
9. Juan's owner cannot see Tala's orders, products or customers, by screen or by direct database query.
10. The storefront installs on Android and iPhone and opens full screen from the home screen.

14. Risks and limitations

Risk	Handling
Scope grows again	Section 4 is the launch list. New ideas go to section 11 until after launch.
Fake payment screenshots	Unique reference numbers, the owner checks their own wallet app, every approval is logged
Owner misses an order	Chime and realtime on the dashboard; Messenger message from the customer; new-order email right after launch
iPhone install friction	Install guide with steps, and an install prompt after the first order
Email codes not delivered	Resend SMTP set up before launch; Supabase built-in email is not used in production
Data leaking between shops	Row-level security on every table and a test that tries cross-shop reads
The repo gets messy again	Fixed folders and import rules enforced by lint (section 8.3)
One-person business	This paper, CLAUDE.md and small checkpoints keep work resumable; daily backups

15. Key decisions

#	Decision	Reason
D1	Two apps: Next.js storefront and Vite dashboard, sharing three packages (ui, core, api)	The storefront needs fast server pages; the dashboard ships to iOS and Android later with Capacitor for printers and the POS. Separating now avoids a move later.

#	Decision	Reason
D2	No admin screen at launch; the founder uses SQL	Saves building a whole surface used by one person
D3	No POS, kitchen display or staff at launch; Messenger hand-off instead	Customers order on their own phones; the owner works from one Orders board
D4	Members and stamps are in the launch version	They are the selling point and the reason to install the app
D5	A member is a customer with member_since set (linked login)	Simple to count and to seed for demos
D6	Money, stock and stamps change only in database functions; simple owner edits use row-level security write rules	Safe where it matters, fast to build elsewhere
D7	Demo data lives in SQL in the database	Reset in seconds from the SQL editor, with no AI tokens
D8	Shop look is logo, cover and three colours (60/30/10)	Enough to feel like the shop's own app; the editor comes later
D9	₱599 per month early plan	Easy yes for small shops; tools break even at about 5 shops
D10	Same database design as the first build, trimmed	Proven; later features plug in without renaming tables
D11	Fixed places for code: apps (screens), ui (design), core (rules), api (data), supabase (database)	Every person and AI agent knows where code goes; lint enforces it

Appendix A: Founder operations by SQL

Run these in the Supabase SQL editor. Each is a saved snippet.

Shops waiting for approval

```
select t.slug, t.name, t.business_type, t.created_at, u.email
from tenants t
join tenant_members m on m.tenant_id = t.id and m.role = 'owner'
join auth.users u on u.id = m.user_id
where t.status = 'pending'
order by t.created_at;
```

Approve a shop with a first month paid

```
update tenants
set status = 'active', plan = 'early_599', paid_until = current_date + interval '1 month'
where slug = 'juans-cafe';
```

Record a monthly payment (extends from the later of today or the current paid_until)

```
update tenants
set paid_until = greatest(current_date, paid_until) + interval '1 month'
where slug = 'juans-cafe';
```

Shops due or overdue

```
select slug, name, paid_until, current_date - paid_until as days_overdue
from tenants
where status = 'active' and paid_until < current_date + 3
order by paid_until;
```

Suspend or restore

```
update tenants set status = 'suspended' where slug = 'some-shop';
update tenants set status = 'active'    where slug = 'some-shop';
```

Usage at a glance

```
select t.slug,
  (select count(*) from orders o where o.tenant_id = t.id and o.placed_at > now() - interval
'30 days') as orders_30d,
  (select count(*) from customers c where c.tenant_id = t.id and c.member_since is not null)
as members
from tenants t order by orders_30d desc;
```

Appendix B: Demo data

Yes, demo data can live directly in Supabase, with no AI tokens.

1. **seed.sql** creates the two test shops, their settings, hours, payment accounts, categories, products, variants and add-ons. It runs automatically with *supabase db reset* on a local database, or can be pasted once into the SQL editor of the hosted project.
2. **demo.sql** adds one function, *demo_reset(shop_slug)*, that deletes only rows marked as demo and creates fresh ones relative to *now()*:
    - about 120 customers (about 75 members)
    - 30 days of orders with an upward trend
    - 3 orders in progress, 1 payment to verify, 2 orders for tomorrow
    - stamps

    Because dates are relative, running it any morning makes the shop look live for that day.
3. Daily use: open the SQL editor and run *select demo_reset('juans-cafe'); select demo_reset('tala-apparel');*
4. Safety:
    - the function refuses to run unless the shop has *is_demo = true*
    - demo rows carry a marker (order token starting with demo-, customer tag demo)
    - clients cannot call it (execute is granted to no one; only the SQL editor can run it)
5. For the front-end phase, the same demo shapes exist as typed mock files in *packages/api*, so screens can be built before the database is connected.

Appendix C: Glossary

Term	Meaning
Tenant / shop	One business account. The code says tenant; screens say shop.
Storefront	The customer app of one shop
Dashboard	The owner app; later also a native iOS and Android app
PWA	A website that installs like an app and opens full screen from the home screen
Capacitor	A tool that wraps a web app as a native iOS or Android app with access to Bluetooth, camera and push
Fulfillment	How the order reaches the customer: walk-in, pickup, delivery
Member	A customer who joined with an email code and can collect stamps
Suki	A regular customer; "Top suki" are the three members who spent the most this month
Row-level security (RLS)	Database rules that decide which rows each signed-in user can read or change
Slug	The shop's short link name, for example juans-cafe
Centavos	Money stored as whole numbers: ₱1.00 = 100
60/30/10	A colour rule: 60% main colour, 30% secondary, 10% accent